Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 13:

- Representing strings in C
- Strings as arguments and return values in C

NPTEL

A CSR Initiative by
kotak
Kotak Mahindra Bank

Python to C: Example 1

- Python code vs. C code

```c
#include <stdio.h>
#include <stdbool.h>

bool is_upper(char c) {
    return 'A' <= c <= 'Z';
}
```

```
Compiler warnings:
gcc-9 -std=gnu17 -Wall -Wno-unused-variable -Wno-unused-but-set-variable -Wno-main

 In function 'is_upper':
5:19: warning: comparison of constant '90' with boolean expression is always true [-Wbool-compare]
    5 |   return 'A' <= c <= 'Z';
      |                   ^~
5:14: warning: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-Wparentheses]
    5 |   return 'A' <= c <= 'Z';
      |          ~~~~^~~~
```

```c
}
```

```python
def is_upper(c: str) -> bool:
    return 'A' <= c <= 'Z'

if __name__ == '__main__':
    print(is_upper('b'))
```

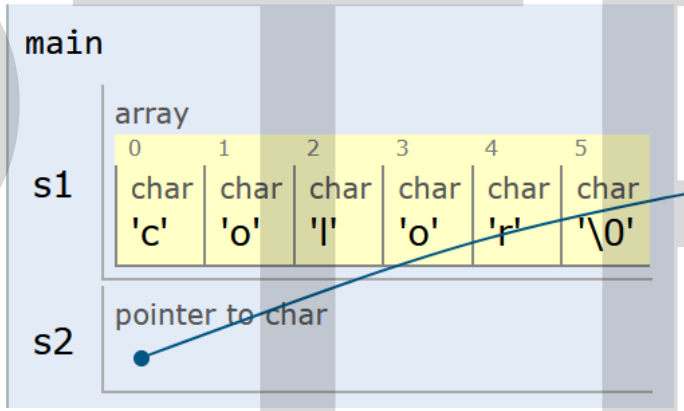
Strings in C

1. Initialize a contiguous chunk of memory at base address ***A***
2. Declare a variable of type `char *` (**pointer** to `char`) and initialize it to ***A***

C (C17 + GNU extensions)
known limitations

```
1  int main() {
2      char s1[] = "color";
3      char *s2 = "colour";
4      return 0;
5  }
```

Edit this code



Declaring an array:

To find the length of a C string, start at the **base address**

Mutable?
PythonTutor

Python list	C array
Heterogeneous	Homogeneous
Automatically resize; `len`	No resize (or: manually); no `len`
Indices: `[-n, …, n-1]`	Indices: `[0, …, n-1]`

Python to C: Example 2

- Python code vs. (incomplete) C code

```python
def to_ind_lower(word: str) -> str:
    """Converts US English spelling to
    lowercase Indian English spelling
    >>> to_ind_lower('color')
    'colour'
    >>> to_ind_lower('FAVORITE')
    'favourite'
    >>> to_ind_lower('Analyze')
    'analyse'
    """
    word = word.lower()
    if word == 'color':
        return 'colour'
    if word == 'favorite':
        return 'favourite'
    if word == 'analyze':
        return 'analyse'
    return word
```

NPTEL

Critique this translation

- Converting to lowercase

```
int i = 0;
while (word[i] != '\0') {
  if (word[i] >= 'A' && word[i] <= 'Z') {
    word[i] += ('a' - 'A');
  }
  i++;
}
```

Same as `i += 1`?

```
def to_ind_lower(word: str) -> str:
    """Converts US English spelling to
    lowercase Indian English spelling
    >>> to_ind_lower('color')
```

```
char *to_ind_lower(const char *word) {
  /* Converts US English spelling to
     lowercase Indian English spelling
     to_ind_lower("color") returns "colour"
     to_ind_lower("FAVORITE") returns "favourite"
     to_ind_lower("Analyze") returns "analyse"
  */
  int i = 0;
  while (word[i] != '\0') {
    if (word[i] >= 'A' && word[i] <= 'Z') {
      word[i] += ('a' - 'A');
  return word
```

Copy the string: Attempt 1

- PythonTutor

```c
char *word_lower = word;
int i = 0;
while (word[i] != '\0') {
  if (word[i] >= 'A' && word[i] <= 'Z') {
    word_lower[i] += ('a' - 'A');
  }
  i++;
}
```

```
Compiler warnings:
gcc-9 -std=gnu17 -Wall -Wno-unused-variable -Wno-unused-but-set-variable -Wno-main

 In function 'to_ind_lower':
10:22: warning: initialization discards 'const' qualifier from pointer target type
   10 |   char *word_lower = word;
      |                      ^~~~
```

NPTEL

Copy the string: Attempt 2

word	'C'	'o'	'l'	'o'	'r'	'\0'
word_lower	'c'	'o'	'l'	'o'	'r'	?

- PythonTutor

```
10    char word_lower[strlen(word) + 1];
11    int i = 0;
12    while (word[i] != '\0') {
13      if (word[i] >= 'A' && word[i] <= 'Z') {
14        word_lower[i] = word[i] + ('a' - 'A');
15      } else {
16        word_lower[i] = word[i];
17      }
18      i++;
19    }
20    if (strcmp(word_lower, "color") == 0) {
```

length array
ed on the **stack**)

NPTEL

Copy the string: Attempt 3

- PythonTutor

```
char word_lower[strlen(word) + 1];
int i = 0;
while (word[i] != '\0') {
  if (word[i] >= 'A' && word[i] <= 'Z') {
    word_lower[i] = word[i] + ('a' - 'A');
  } else {
    word_lower[i] = word[i];
  }
  i++;
}
word_l
if (st
```

```
Compiler warnings:
gcc-9 -std=gnu17 -Wall -Wno-unused-variable -Wno-unused-but
 In function 'to_ind_lower':
30:10: warning: function returns address of local variable
   30 |   return word_lower;
      |          ^~~~~~~~~~
```

NPTEL

Copy the string: Attempt 4

- PythonTutor

Request the **Operating System**

Please allocate so many **bytes** of **contiguous** memory on the **heap**



What if the request is denied? Who will de-allocate (free) this memory?

```
char *word_lower = malloc(strlen(word) + 1);
int i = 0;
while (word[i] != '\0') {
  if (word[i] >= 'A' && word[i] <= 'Z') {
    word_lower[i] = word[i] + ('a' - 'A');
  } else {
```

```
Compiler warnings:
gcc-9 -std=gnu17 -Wall -Wno-unused-variable -Wno-unused-but-set-variable -Wno-

 In function 'to_ind_lower':
10:22: warning: implicit declaration of function 'malloc' [-Wimplicit-function
   10 |    char *word_lower = malloc(strlen(word) + 1);
      |                       ^~~~~~
10:22: warning: incompatible implicit declaration of built-in function 'malloc
2:1: note: include '<stdlib.h>' or provide a declaration of 'malloc'
    1 | #include <string.h>
  +++ |+#include <stdlib.h>
```

Summary

- C strings are null-terminated arrays of `char`
  - Several helper functions in `string.h` (e.g., `strlen`, `strcmp`, `strcpy`, …)
  - If you need to process strings, use Python
- An array is a contiguous chunk of memory starting at a **base address**
  - For a function with an array argument, this base address is copied (***not*** the whole array)
  - We cannot return the address of a local array
- Allocating arrays: fixed-length (stack), variable-length (stack), `malloc` (heap)
  - The programmer must handle heap memory in C

NPTEL